

06 · Explicación profunda del código

Documentacion del sistema

Version analizada: **0.3.0**

Commit analizado: **ff246ab**

Fecha de generacion: **2026-08-27**

Generado a partir de 06-deep-code-explanation.md — los Markdown son la fuente unica.
No editar este PDF: editar el Markdown y regenerar.

Contenido de este documento

1. ``engine/orchestrator.py`` — el corazón
2. ``engine/template.py`` — la sustitución de placeholders
3. ``engine/sandbox.py`` — la política
4. ``engine/conditions.py`` — la evaluación
5. ``engine/loader.py`` — del JSON a las dataclasses
6. ``engine/database.py`` — la persistencia
7. ``actions/browser_extract.py`` — el módulo mejor diseñado
8. ``actions/browser_form.py`` — la operación más compleja
9. ``engine/cron.py`` — el planificador propio
10. ``engine/scheduler.py`` — el bucle de disparo
11. ``app/server.py`` — el panel
12. ``engine/introspection.py`` — qué cuenta como salida
13. ``plugins/analyzers/ocr_image_analyzer.py`` — degradación bien hecha
14. ``scripts/validate_project.py`` — el gate

Flujo interno módulo a módulo: qué entra, qué sale, qué decide, dónde falla y qué casos límite hay. Se explica por bloques lógicos donde la literalidad sería inútil, y línea a línea donde el detalle importa. Cada fragmento cita archivo y símbolo.

1. engine/orchestrator.py — el corazón

Objetivo. Convertir un `manifest.json` en una corrida trazable. **Entrada.** Carpeta del flow, ruta opcional de contexto, diccionario opcional de overrides. **Salida.** El diccionario `state` completo, y tres almacenes persistentes.

1.1 Construcción

```
def __init__(self, flow_dir, context_path=None, context_overrides=None):
    init_db()
    self.definition = FlowLoader.load_manifest(flow_dir)
    self.context = FlowLoader.load_context(flow_dir, context_path)
    if context_overrides:
        self.context = {**self.context, **context_overrides}
```

Cinco hechos que conviene tener presentes:

1. `init_db()` en el constructor. Crear un `Orchestrator` crea `db/runs.db` y sus siete tablas. No hay forma de instanciarlo sin tocar el disco. Es lo que permite que el CLI, el panel, el scheduler y el smoke test compartan inicialización sin coordinarse.
2. Los overrides son una fusión superficial. `{**base, **overrides}` reemplaza claves de primer nivel completas. Un override de `{"bbox": {"top": -48}}` borra el resto de claves de `bbox`. No hay fusión profunda.
3. La política se construye desde la definición, no desde el manifest crudo. El `FlowLoader` ya normalizó los campos, así que `SandboxPolicy.from_manifest` —que existe— no se usa aquí; el orquestador instancia `SandboxPolicy(...)` directamente con los campos de `FlowDefinition`.
4. El `run_id` se genera en el constructor, no en `run()`. Por eso el panel puede instanciar, persistir y devolver el `run_id` al cliente antes de que la corrida empiece.
5. `log_path` y `state_path` se construyen sobre `data_dir()`, no sobre `root_dir()`. Es lo que hace que el binario empaquetado escriba en `%LOCALAPPDATA%` en vez de en `Program Files`.

El `state` inicial trae 16 claves, incluidas `status: 'created'`, `steps: []`, `outputs: []`, `route: []` y `error: None`.

1.2 run() — bloque por bloque

Bloque 1 • secretos. Antes de nada, `self.policy.assert_secrets_present()`. Si falta alguno, el flow se marca `failed` con `kind: 'sandbox_violation'`, se persiste, se escribe el evento `flow_blocked` y se levanta `FlowExecutionError`. Es el único control que corre una vez por corrida y no por paso.

Bloque 2 • arranque. `status = 'running'`, `started_at`, `state['policy'] = policy.summary()`. Ese `summary()` incluye `permissive: bool`, así que el histórico registra si la corrida tenía sandbox o no. Es una decisión de auditabilidad excelente y poco visible: consultando `runs.context_json` se puede saber, corrida a corrida, cuáles corrieron sin restricción.

Bloque 3 • el bucle.

```
current_step_id = self.definition.start_step or (self.step_order[0] if self.step_order else None)
executed_count = 0
while current_step_id:
    if executed_count >= self.definition.max_steps_per_run:
        raise FlowExecutionError('Se alcanzó el máximo de pasos permitidos para esta corrida.')
    if current_step_id == '__END__':
        break
```

`__END__` es el centinela que devuelve `_resolve_transition` cuando una transición lleva `"end": true`. No es un paso real; nunca aparece en `steps_by_id`.

`max_steps_per_run` (200 por defecto) es la protección contra ciclos infinitos. Un flow con transiciones circulares no cuelga el proceso: se detiene con error tras 200 pasos. **Nota:** el contador cuenta *ejecuciones de paso*, no pasos distintos, así que un bucle de dos pasos se corta a las 200 iteraciones.

Bloque 4 • condición del paso.

```
if step.when and not evaluate_condition(step.when, self.context):
    step_record = {..., 'status': 'skipped', 'result': {'reason': 'condition_not_met'}, ...}
```

Un paso saltado **sí se registra** en `state['steps']` y en la tabla `steps`, con `attempt: 0` y `duration_seconds: 0.0`. Es lo que permite al panel distinguir «no se ejecutó por condición» de «no llegó a ejecutarse». Después se resuelve la transición con evento `success` —un salto se considera éxito, no fallo— y `executed_count` **sí se incrementa**, contando contra `max_steps_per_run`.

Bloque 5 · los tres controles previos, en orden estricto.

```
self.policy.assert_action_allowed(step.action)          # 1
rendered_params = render_value(step.params, self.context)
self.policy.assert_paths_allowed(rendered_params)      # 2
action = ACTION_REGISTRY.get(step.action)
if action is None:
    raise FlowExecutionError(f'Acción no registrada: {step.action}')
if self.policy.max_runtime_seconds is not None:        # 3
    elapsed = (datetime.now(timezone.utc) - started_at).total_seconds()
    if elapsed > self.policy.max_runtime_seconds:
        raise FlowExecutionError(...)
```

El orden importa y está bien elegido: la acción se valida antes de renderizar (más barato), y las rutas se validan **después** de renderizar, porque un `{output_dir}` sin resolver no se puede comprobar. `assert_paths_allowed` además descarta explícitamente cualquier cadena que aún contenga `{`.

Caso límite del tiempo: `max_runtime_seconds` se compara con el tiempo transcurrido **antes** de lanzar el paso. Si un paso tarda diez minutos, el límite de 30 s no lo interrumpe: se detecta al ir a por el siguiente paso, y si era el último, la corrida termina `completed` habiendo excedido el límite. Es un control de *arranque de paso*, no un timeout real.

Bloque 6 · el bucle de reintentos.

```
attempts = 0
while attempts <= step.retries:
    attempts += 1
    try:
        result = action(**rendered_params)
        ...
        break
    except Exception as exc: # noqa: BLE001
        last_error = str(exc)
        self.context['_last_error'] = {'message': last_error, 'step_id': step.id}
        if attempts > step.retries:
            ...
```

Tres detalles no evidentes:

- **rendered_params** se calcula una sola vez, fuera del bucle. Un `{now}` en el nombre de archivo produce el mismo nombre en todos los reintentos.
- **No hay espera entre reintentos.** Tres intentos consecutivos contra un servidor caído ocurren en milisegundos. Para reintentos con espera hay que meter un paso `system.wait_seconds` en el manifest.
- **_last_error** queda en el contexto aunque el paso acabe teniendo éxito en un intento posterior. Una condición que consulte `_last_error` puede leer un error viejo.

Bloque 7 · la recuperación. Cuando se agotan los reintentos:

```
recovery_next = self._resolve_transition(step, event='failure')
if recovery_next and recovery_next != self._default_next(step.id):
    self.logger.write('step_recovered', {...})
    current_step_id = recovery_next
    self._persist()
    break
self.state['status'] = 'failed'
```

Esta comparación es todo el mecanismo de recuperación. `_resolve_transition` devuelve `_default_next` cuando ninguna transición coincide, así que la única forma de distinguir «hay rama de recuperación» de «no hay ninguna» es comprobar que el destino sea distinto del siguiente por defecto.

Consecuencia sutil: un manifest que declare `{"on": "failure", "next": "<el paso siguiente>"}` —es decir, que quiera continuar por el camino normal tras un fallo— **no se recupera**, porque el destino coincide con el por defecto y la comparación falla.

Hay que apuntar a un paso distinto. No está documentado en [docs/CREAR_FLUJOS.md](#).

Bloque 8 · cierre. El `except Exception` externo persiste el estado con `finished_at` y `duration_seconds` y escribe `flow_finished` antes de re-lanzar. Así, **incluso una corrida fallida deja duración medida y evento de cierre**. En el camino feliz, `status = 'completed'` y se devuelve el `state`.

1.3 `_resolve_transition`

```
for transition in step.transitions:
    if transition.on not in {event, 'any'}:
        continue
    if transition.when and not evaluate_condition(transition.when, self.context):
        continue
    if transition.end:
        return '__END__'
    if transition.next_step:
        return transition.next_step
return self._default_next(step.id)
```

Primera coincidencia gana. Una transición con `on` compatible y `when` cumplida pero **sin `next` ni `end`** no devuelve nada y el bucle sigue a la siguiente: es una transición inerte, y el validador no la detecta.

1.4 `_persist` y `_refresh_outputs`

```
def _persist(self):
    self._refresh_outputs()
    self.state_store.save(self.state)
    upsert_run(self.state, self.flow_dir.name, str(self.state_path), str(self.log_path))
```

Se llama tras **cada** paso. Cada llamada recorre todo el `state` buscando rutas de archivo (`extract_existing_paths`), reescribe el JSON completo de `state/` y hace un `INSERT ... ON CONFLICT DO UPDATE` en `runs`. Para un flow de tres pasos son tres recorridos completos y tres escrituras. **INFERENCIA:** en un flow de 200 pasos con un contexto grande, el costo de `_persist` domina el tiempo de ejecución.

2. `engine/template.py` — la sustitución de placeholders

Objetivo. Resolver `{clave}` y `{objeto.campo}` dentro de los `params` del manifest.

2.1 `flatten_context`

Convierte `{"a": {"b": 1}}` en `{"a": {"b": 1}, "a.b": 1}`. Conserva **ambas** claves, así que `{a}` devuelve el diccionario entero y `{a.b}` devuelve `1`.

2.2 La decisión clave: placeholder exacto frente a placeholder embebido

```
def render_value(value, context):
    if isinstance(value, str):
        prepared = value.replace("{", "{").replace("}", "}")
        flat = flatten_context(context)
        flat["now"] = datetime.now().strftime("%Y%m%d_%H%M%S")
        exact = _resolve_exact_placeholder(prepared, flat)
        if exact is not _MISSING:
            return exact
        return _substitute_placeholders(prepared, flat)
```

Si la cadena es **exactamente** un placeholder, se devuelve el valor **con su tipo original**. Por eso `"headless": "{ headless }"` en el manifest 07 llega a `fill_form` como el booleano `False`, no como la cadena `"False"`.

El centinela `_MISSING` distingue «la clave no existe» de «la clave existe y vale `None`». Sin él, un placeholder que resuelve a `None` caería al render de string y produciría `"None"` en vez de `null`. El módulo lo documenta.

2.3 Por qué no se usa `str.format_map`

```
def _replace(match):
    key = match.group(1).strip()
    if key in flat:
        return str(flat[key])
    return match.group(0) # queda literal
```

Dos razones, ambas en el docstring del módulo:

1. `str.format_map` interpreta el punto como **acceso a atributo**, así que `"{content.content_hash}"` lanzaría `AttributeError` sobre un dict. Con la sustitución manual sobre el contexto aplanado, funciona.
2. Una llave que no es placeholder —JSON embebido en un comando, por ejemplo— haría crashear a `format_map`. Aquí queda intacta.

El `CHANGELOG.md` de la v0.3.0 confirma que esto fue un cambio de comportamiento real: antes, un mensaje de `notify` con `{content.content_hash}` reventaba.

{now} es especial: se inyecta en cada llamada a `render_value`. Dentro de un mismo paso todos los `{now}` coinciden, pero **entre pasos pueden diferir** si el segundo cambia. Un flow que escriba una imagen en el paso 1 y la referencie en el paso 2 con otro `{now}` fallaría. Los flows del repositorio evitan el problema pasando la ruta por el contexto con `save_as`.

3. `engine/sandbox.py` — la política

3.1 Las cuatro dimensiones

Método	Cuándo	Qué comprueba
<code>assert_secrets_present</code>	Una vez, al arrancar	Que cada nombre de <code>required_secrets</code> esté en <code>os.environ</code> y no vacío
<code>assert_action_allowed</code>	Antes de cada paso	Que <code>step.action</code> esté en <code>allowed_actions</code> , si la lista existe
<code>assert_paths_allowed</code>	Antes de cada paso, sobre params renderizados	Que toda ruta candidata caiga bajo alguna base
<code>max_runtime_seconds</code>	Entre pasos, en el orquestador	Tiempo total transcurrido

`check_required_secrets` lee **solo** `os.environ`, no `engine.secrets.get_secret`. Un secreto que viva únicamente en `secrets/secrets.json` **no satisface** `required_secrets`. Es una inconsistencia entre dos mecanismos que se presentan como equivalentes. **NO DOCUMENTADO EN EL REPOSITORIO.**

3.2 `_path_strings` — la heurística de detección de rutas

```
if any(token in key.lower() for token in ('path', 'destination', 'source', 'output', 'file')):
    if isinstance(value, str):
        yield value
    yield from self._path_strings(value)
```

Detecta rutas **por el nombre de la clave**, no por la forma del valor. Consecuencias verificables:

- [OK] `output_path`, `save_data_path`, `seeds_path`, `destination_path`, `state_path`, `track_state_path` → se comprueban.
- [NO] `target` de `browser.extract_content` y `browser.capture_page` → **no se comprueba**, aunque puede ser una ruta local a un `.html`.
- [NO] `command` de `ui.launch_process` → **no se comprueba**, aunque contiene una ruta.

En el catálogo actual no genera un agujero, porque los siete flows con `allowed_paths` no usan esas acciones. Pero es una limitación que un autor de flows debe conocer. Registrada en [11 · Seguridad](#).

Además, `yield from self._path_strings(value)` recorre el valor **aunque la clave ya haya coincidido**, lo que significa que un dict anidado bajo una clave `path` se recorre dos veces. Sin consecuencia funcional, solo trabajo duplicado.

3.3 is_permissive y summary

`is_permissive()` devuelve `True` cuando las cuatro dimensiones están vacías. `summary()` lo incluye, y el orquestador lo guarda en `state['policy']`. Así queda registrado en el histórico si la corrida tenía o no restricciones. Para los 14 flows sin política, el histórico dirá `permissive: true`.

4. engine/conditions.py — la evaluación

4.1 get_path navega con tolerancia

```
for part in path.split('.'):
    if isinstance(current, dict):
        current = current.get(part, default)
    else:
        return default
```

Un camino que atraviesa una lista devuelve `default (None)`. No hay acceso por índice: `steps.0.status` no funciona. Es una limitación real del lenguaje de condiciones.

4.2 Los operadores y sus trampas

- `gt`, `gte`, `lt`, `lte` comprueban `actual is not None` antes de comparar, así que nunca lanzan `TypeError` por comparar con `None`. Pero **sí lo lanzarían** al comparar una cadena con un número (`"80" > 70`). Un contexto que traiga el umbral como texto falla en tiempo de ejecución.
- `contains` normaliza a minúsculas **ambos lados**: `str(expected).lower() in str(actual).lower()`. Es una búsqueda insensible a mayúsculas por diseño.
- `in` devuelve `False` si `expected` no es una lista, sin avisar.
- `regex` usa `re.search`, no `re.fullmatch`: basta con que el patrón aparezca en algún punto.

4.3 Los combinadores

`all` sobre lista vacía devuelve `True`; `any` sobre lista vacía devuelve `False`. Es la semántica de Python y probablemente la esperada, pero un manifest con `{"all": []}` como guarda deja pasar siempre.

5. engine/loader.py — del JSON a las dataclasses

5.1 load_manifest

Lee el JSON **sin validarlo contra el schema**. Los campos obligatorios se acceden con corchetes (`raw['id']`, `raw['steps']`, `step['id']`, `step['action']`), así que un manifest incompleto produce un `KeyError` crudo, no un mensaje del validador.

`allowed_actions` y `allowed_paths` usan `if raw.get(...)` en vez de `is not None`, de modo que **una lista vacía se convierte en None**, es decir, en «sin restricción». Escribir `"allowed_actions": []` con la intención de bloquear todo produce el efecto contrario: política permisiva. Es una trampa real. Registrada en 15.

5.2 load_context — la precedencia

```
candidates = []
if explicit_context_path:
    candidates.append(explicit_context_path)
candidates.extend([
    Path('configs') / f'{flow_dir.name}.json',
    flow_dir / 'context.user.json',
    flow_dir / 'context.example.json',
])
for candidate in candidates:
    if candidate.exists():
        return json.load(fh)
return {}
```

La primera que exista gana y las demás se ignoran por completo. No hay mezcla de claves. Editar la configuración desde el panel crea `configs/<folder>.json`, que a partir de ese momento **oculta** el `context.example.json`: si una versión posterior del flow añade una clave nueva al ejemplo, el flow configurado no la verá y el parámetro llegará sin resolver.

Trampa adicional: `Path('configs')` es una ruta **relativa al directorio de trabajo**, no a `root_dir()` ni a `data_dir()`. Ejecutar `automa run` desde otra carpeta hace que `configs/` no se encuentre. Contrasta con `set_flow_config`, que sí escribe en `data_dir() / 'configs'`. **Son dos rutas distintas** cuando el binario corre congelado: el panel escribiría en `%LOCALAPPDATA%\Automa\configs` y el loader leería `./configs`. Registrado en 15.

6. engine/database.py — la persistencia

6.1 El patrón de conexión

```
@contextmanager
def connect():
    DB_PATH.parent.mkdir(parents=True, exist_ok=True)
    conn = sqlite3.connect(DB_PATH)
    conn.row_factory = sqlite3.Row
    try:
        yield conn
        conn.commit()
    finally:
        conn.close()
```

Conexión nueva por operación, `commit` solo si no hubo excepción, `close` siempre. Evita el problema de compartir conexiones entre hilos. El costo es abrir el archivo en cada consulta.

NO IDENTIFICADO: no se configura `journal_mode=WAL` ni `busy_timeout`. Con el panel lanzando corridas en paralelo y el scheduler escribiendo a la vez, `database is locked` es posible. **REQUIERE VALIDACIÓN** bajo carga.

6.2 init_db y la migración

`executescript` con siete `CREATE TABLE IF NOT EXISTS`. Después, una única migración:

```
_ensure_column(conn, 'schedules', 'cron_expression', 'cron_expression TEXT')
```

`_ensure_column` consulta `PRAGMA table_info` y añade la columna solo si falta. Es toda la maquinaria de migración del proyecto: suficiente para el cambio que hubo, insuficiente para un cambio de tipo o un renombrado.

6.3 acquire_run_lock — el lock que casi nadie usa

```
try:
    conn.execute('INSERT INTO run_locks(folder, run_id, acquired_at) VALUES(?,?,?)', ...)
    return True
except sqlite3.IntegrityError:
    return False
```

La exclusión mutua se apoya en la **clave primaria folder** de `run_locks`: el segundo insert viola la restricción y se traduce en `False`. Es correcto y elegante.

Pero solo lo usa SchedulerService._run_job. Verificado buscando en todo el repositorio: `POST /api/run`, `POST /run`, `POST /api/hook` y el CLI **no adquieren el lock**. El panel sí permite lanzar el mismo flow dos veces en paralelo.

Y no hay liberación al arrancar. Si el proceso muere con un lock tomado, la fila queda para siempre y el scheduler no volverá a disparar ese flow. El `RUNBOOK.md` documenta la liberación manual con `force_release_lock`, lo que confirma que el escenario ocurre.

6.4 set_flow_config escribe en dos sitios

```
config_path = data_dir() / 'configs' / f'{folder}.json'
config_path.write_text(json.dumps(config, ensure_ascii=False, indent=2), encoding='utf-8')
conn.execute('INSERT INTO flow_configs(...) ON CONFLICT(folder) DO UPDATE ...')
```


Archivo y tabla. El archivo es lo que lee `FlowLoader.load_context`; la tabla es lo que lee `get_flow_by_folder` para mostrarlo en el panel. **No hay transacción que abarque ambos**: si el `write_text` tiene éxito y el `INSERT` falla, quedan desincronizados.

`write_text` sin `newline='\n'` produce CRLF en Windows. Como `configs/03_folder_inventory.json` está versionado y el smoke test lo reescribe, ejecutar el smoke test puede dejar el árbol de trabajo marcado como modificado.

7. `actions/browser_extract.py` — el módulo mejor diseñado

567 líneas divididas en dos mitades con una frontera explícita, documentada en el propio docstring: «la interacción con Playwright vive en `scrape_page/extract_content/crawl_site` (glue mínimo), y toda la lógica son funciones puras testeables sin navegador».

7.1 `normalize_text` — por qué el hash es estable

```
lines = [line.strip() for line in (text or '').splitlines()]
collapsed = []
for line in lines:
    if line or (collapsed and collapsed[-1]):
        collapsed.append(line)
while collapsed and not collapsed[-1]:
    collapsed.pop()
return '\n'.join(collapsed)
```

Recorta cada línea, colapsa líneas vacías consecutivas en una sola y quita las finales. Sin esto, un cambio de espaciado en el HTML cambiaría el SHA-256 y el flow 23 reportaría un cambio falso.

Riesgo de tocarla: cualquier modificación invalida **todos** los hashes ya guardados en `data/web_watch/*.json`. La primera corrida posterior reportaría `changed: true` en todas las páginas vigiladas.

7.2 `parse_number` — la heurística documentada

```
if ',' in cleaned and '.' in cleaned:
    dec = max(cleaned.rfind(','), cleaned.rfind('.')) # el separador más a la derecha
elif ',' in cleaned or '.' in cleaned:
    if len(parts) == 2 and (len(parts[1]) != 3 or parts[0] in ('', '0')):
        value = float(...) # es decimal
    else:
        value = float(''.join(parts)) # es separador de miles
```

Convierte `$1,499.90` y `1.499,90 €` al mismo `1499.9`. La regla con un solo separador: si detrás hay **exactamente tres dígitos**, se asume separador de miles, **salvo** que la parte entera sea vacía o `0`.

Casos límite reales: `"1.234"` → `1234.0` (interpretado como miles). Si el valor vigilado fuera realmente `1.234` como decimal, el flow 26 leería `1234.0` y cruzaría cualquier umbral. La heurística está documentada en el docstring; es una decisión, no un descuido, pero un usuario debe conocerla.

7.3 `resolve_links` — orden y deduplicación

Preserva el orden de aparición en el DOM, deduplica por URL absoluta, descarta anclas (`#`), `javascript:` y `data:`, y **conserva** `mailto:` y `tel:` sin resolver contra la base. Corta en `max_links` y devuelve `truncated`. `urldefrag` elimina el fragmento, de modo que `pagina#a` y `pagina#b` cuentan como el mismo enlace.

7.4 `apply_tracking` — el detector de cambios

```
first_run = previous is None
previous_value = (previous or {}).get('watch_value')
changed = (not first_run) and previous_value != watch_value
```

La primera corrida nunca reporta cambio, aunque el archivo de estado no exista. Establece la línea base y devuelve `first_run: True`. Es lo que evita una notificación espuria al estrenar el flow.

Un archivo de estado corrupto se trata como inexistente (`except (json.JSONDecodeError, OSError): previous = None`), lo que reinicia la línea base **en silencio**: el flow diría `first_run: true` otra vez sin explicar por qué. Es el único punto del módulo donde una degradación no se declara.

7.5 crawl_pages — BFS acotado

```
queue = deque([(start_url, 0)])
seen = {start_url}
while queue and len(pages) < max_pages:
    url, depth = queue.popleft()
    if robots_check is not None and not robots_check(url):
        robots_blocked.append(url); continue
    try:
        raw = fetch_page(url)
    except Exception:
        errors.append({'url': url, 'error': str(exc)}); continue
```

Cuatro decisiones visibles:

1. **Un enlace caído no aborta el crawl.** Se registra en `errors` y se sigue.
2. **seen se marca al encolar, no al visitar,** evitando duplicados en la cola.
3. **El filtro de dominio compara netloc exacto.** `www.example.com` y `example.com` son dominios distintos.
4. **truncated es bool(queue):** dice si quedaron URLs sin visitar al alcanzar el tope. Honesto.

`robots_check` se aplica **antes** de la petición, incluida la URL inicial: un `robots.txt` que prohíba la raíz produce un crawl de cero páginas con `robots_blocked` de uno.

7.6 RobotsCache — el fallo permisivo declarado

```
except requests.RequestException:
    self.checked_hosts[host] = False
    return None          # parser None -> allows() devuelve True
```

Si `robots.txt` no se puede leer, **se permite el acceso**, pero se registra `checked=False` en `robots_checked_hosts`, que va al reporte del flow 22. El docstring lo declara: «se permite el acceso pero se registra `checked=False` para reportarlo con honestidad». Un 4xx sí cuenta como comprobado (no hay reglas) y también permite.

7.7 crawl_site y la recuperación de página rota

```
def fetch_page(page_url):
    nonlocal page
    try:
        page.goto(page_url, wait_until='load')
    except Exception:
        try: page.close()
        except Exception: pass
        page = new_page()
        raise
```

Un `goto` fallido deja la pestaña de Playwright en estado de error que rompería la navegación siguiente. La solución: cerrar la pestaña, abrir una limpia y **propagar** el error para que `crawl_pages` lo registre. El comentario del código lo explica. Es el tipo de detalle que solo aparece tras ejecutar el crawl contra un sitio real.

8. actions/browser_form.py — la operación más compleja

8.1 _pick_record — tracking sin repetición

```
used = _load_used_ids(used_path)
available_ids = [r['id'] for r in records if r['id'] not in used]
if not available_ids:
    used = set(); available_ids = [r['id'] for r in records]; cycle_resetted = True
chosen_id = random.choice(available_ids)
used.add(chosen_id)
_save_used_ids(used_path, used, len(records))
```

El registro se marca como usado antes de intentar el llenado. Si el navegador falla después, ese registro se pierde para el ciclo. Es una decisión conservadora —prefiere saltarse un registro a repetirlo— pero no está documentada en el README del flow.

El reinicio automático al agotar los 100 se declara con `cycle_resetted: True` en el resultado.

8.2 El llenado

```
for field in ('nombre', 'apellido', 'email', 'telefono', 'direccion',
             'ciudad', 'fecha_nacimiento', 'profesion', 'comentario'):
    page.fill(f'#{field}', form_data[field])
page.select_option('#pais', form_data['pais'])
```

Nueve `fill` más un `select_option`. Los identificadores están **escritos en Python**, no en el manifest: la acción solo sirve para el formulario de demo o para uno con exactamente los mismos `id`. Es la limitación de alcance más importante del flow más vistoso del repositorio.

Un `KeyError` si el registro semilla no trae alguna de las diez claves; el orquestador lo convierte en fallo de paso.

8.3 La validación

```
page.wait_for_selector('#validation-result.show', timeout=5000)
...
except Exception:
    validation_text = '(timeout esperando #validation-result.show)'
```

El timeout **no falla el paso**: deja constancia textual y sigue. Después:

```
is_success = validation_text.startswith('[OK]') or 'válido' in validation_text.lower()
```

El éxito se decide comparando emoji y texto en español. Depende por completo del HTML de demo. Si la página cambia su copy, `is_success` pasa a `False` sin que nada más falle.

9. engine/cron.py — el planificador propio

Objetivo. Calcular la próxima ejecución de una expresión de 5 campos sin dependencias.

9.1 Lo que soporta y lo que no

Soporta `*`, números, listas (`1,3,5`), rangos (`9-17`) y pasos (`*/5, 0-30/10`). **No soporta** nombres simbólicos (`MON, JAN`) ni `L/W/#`. El docstring lo declara y explica la decisión: «Es deliberadamente pequeño: cubre los casos comunes [...] sin meter una dependencia externa».

9.2 El día de la semana no es el estándar

```
(0, 6), # day of week (0=lunes, 6=domingo, estilo ISO)
...
def _iso_weekday(dt): return dt.weekday()
```

En cron estándar, 0 es domingo. Aquí 0 es lunes. Una expresión copiada de `crontab(5)` se ejecutará el día equivocado. Está documentado en el comentario del código, pero **no** en `docs/OPERACION.md` ni en la ayuda del panel. Registrado en 15.

9.3 El avance por saltos

```
for _ in range(60 * 24 * 366 * 4): # ≈ 4 años en minutos
    if candidate.month not in months.values:
        candidate = datetime(year, month, 1, 0, 0, tzinfo=timezone.utc); continue
    if candidate.day not in days.values:
        candidate = (candidate + timedelta(days=1)).replace(hour=0, minute=0); continue
    ...
```

En vez de avanzar minuto a minuto, salta al inicio del mes, del día o de la hora siguientes cuando el campo no coincide. Reduce drásticamente las iteraciones. El límite de cuatro años convierte una expresión imposible (`0 0 30 2 *`, 30 de febrero) en `CronExpressionError` en vez de un cuelgue.

Combinación día-del-mes / día-de-la-semana: el código exige que **ambos** coincidan (AND). Cron estándar usa OR cuando ninguno es `*`. Otra divergencia con `crontab(5)`.

9.4 Todo en UTC

`next_after` trabaja siempre en UTC. Una expresión `0 9 * * 1` se dispara a las 09:00 UTC, no a las 09:00 locales. Para un operador en UTC-3 eso son las 06:00. **NO DOCUMENTADO EN EL REPOSITORIO.**

10. `engine/scheduler.py` — el bucle de disparo

```
def run_pending_once(self):
    for schedule in list_schedules():
        if self._should_run(schedule):
            thread = threading.Thread(target=self._run_job, args=..., daemon=True)
            thread.start()
```

Cada tarea vencida arranca su propio hilo daemon. `_should_run` compara `next_run_at` con `now` y exige `enabled`. Una tarea sin `next_run_at` nunca se dispara.

`_run_job` adquiere el lock, ejecuta y **libera el lock en `finally`**, de modo que una excepción no deja el lock tomado *dentro del proceso vivo*. Lo que sí lo deja es matar el proceso a mitad.

El error se traga por completo:

```
try:
    Orchestrator(Path(flow['flow_path'])).run()
except Exception:
    pass # "No interrumpe el loop. El error queda en la corrida."
```

Y `mark_schedule_run` se llama **después del `except`**, es decir, **también cuando el flow falla**. Consecuencia: una tarea programada que falla siempre reprograma su siguiente ejecución con normalidad y no genera ninguna señal. Solo se descubre mirando el histórico.

11. `app/server.py` — el panel

11.1 Los efectos de importar el módulo

```
SCHEDULER = SchedulerService(loop_sleep_seconds=2.0)
SCHEDULER.start_in_background()
init_db()
sync_flows(list_flows())
```

Cuatro efectos a nivel de módulo: hilo de scheduler vivo, base creada, catálogo sincronizado. Cualquier `import app.server` los dispara, incluido el de los tests.

11.2 `_is_preview` — dos mecanismos

```
if (flow_path / '.disabled').exists():
    return True
manifest = json.loads(...)
return bool(manifest.get('preview'))
```

El archivo marcador `.disabled` permite desactivar un flow **sin tocar el manifest ni commitear**. Es un detalle operativo útil y poco visible. Un manifest ilegible devuelve `False` (flow operativo), decisión discutible: un manifest corrupto se considera ejecutable.

Ningún flow del catálogo está en preview.

11.3 `_run_status_payload` — pending frente a not_taken

```
fallback_status = 'pending' if is_running else 'not_taken'
```

Un paso del manifest sin registro en `steps` significa cosas distintas según el estado de la corrida: si sigue viva, está **pendiente**; si terminó, es una **rama no tomada**. El comentario del código lo explica. Es el detalle que hace que el panel no muestre pasos «pendientes» eternos en corridas ya cerradas.

11.4 /api/run/<folder> — la ejecución asíncrona

```
orch = Orchestrator(Path(flow['flow_path']), context_overrides=overrides or None)
orch.state['status'] = 'running'
orch._persist() # sincrónico, antes del hilo
...
threading.Thread(target=_runner, daemon=True).start()
return self._send_json({'ok': True, 'run_id': run_id, ...})
```

El comentario explica el porqué: «Persistimos sincrónicamente antes de lanzar el thread para garantizar que el polling vea el run desde el primer tick». Sin eso, el navegador podría pedir `/api/runs/<run_id>/status` antes de que la fila existiera.

Nótese que se llama a `orch._persist()`, un método privado, desde fuera de la clase. Es acoplamiento con el interior del orquestador.

11.5 `_authorize_mutation` — el modelo de dos modos

Explicado en 03 §8 y analizado en 11 · Seguridad. El punto de código que importa aquí:

```
if hmac.compare_digest(provided, expected)
```

Comparación en tiempo constante, que cierra CWE-208 (fuga por temporización). Está bien hecho y el comentario lo cita explícitamente.

12. `engine/introspection.py` — qué cuenta como salida

```
if len(value) > 260 or '\n' in value:
    return
try:
    candidate = Path(value)
    if not candidate.exists() or not candidate.is_file(): return
    resolved = candidate.resolve()
    resolved.relative_to(output_root)
except (ValueError, OSError):
    return
```

Tres filtros con motivo explicado en comentarios del propio archivo:

1. **Longitud > 260 o con salto de línea** → no es una ruta. El comentario razona: los paths reales rara vez pasan de 260 caracteres en Windows, y llamar a `Path.exists()` sobre una cadena larguísima produce `OSError ENAMETOOLONG` en Linux.
2. **Debe existir y ser archivo.**
3. **Debe estar bajo `output/`.** El comentario explica que antes se aceptaba cualquier ruta existente y eso contaminaba la lista con archivos que el flow solo había leído.

`_output_root()` usa `Path('output').resolve()`, relativo al directorio de trabajo. Por eso `installer/automa_entry.py` hace `os.chdir(data_dir())` en el bundle: sin ese `chdir`, los outputs no se detectarían.

13. `plugins/analyzers/ocr_image_analyzer.py` — degradación bien hecha

```
@staticmethod
def _tesseract_binary_available() -> bool:
    if shutil.which('tesseract'): return True
    for candidate in ('C:/Program Files/Tesseract-OCR/tesseract.exe',
                      'C:/Program Files (x86)/Tesseract-OCR/tesseract.exe'):
        if Path(candidate).exists(): return True
    return False
```

Busca en el PATH y, además, en las dos rutas donde el instalador de Windows deja el binario sin añadirlo al PATH. Es un detalle que ahorra un caso de soporte muy frecuente.

Cuando falta, devuelve `status: 'unavailable'` con `reason` (`pytesseract_missing` o `tesseract_binary_missing`) y una `summary` con el comando de instalación por sistema operativo. El flow continúa; los pasos siguientes recibirán `matches: []`.

La extracción de coordenadas usa `pytesseract.image_to_data` con `output_type=Output.DICT` y descarta las entradas con texto vacío, quedándose con `text`, `conf`, `left`, `top`, `width`, `height` por palabra. Ese formato es el que consume `ui.click_bbox`.

14. `scripts/validate_project.py` — el gate

Cinco comprobaciones por manifest, en orden:

1. **JSON Schema** (o el respaldo estructural si falta `jjsonschema`). Si falla, se **detiene ahí** para ese flow: no tiene sentido comprobar acciones sobre un manifest inválido.
2. **IDs de paso duplicados**.
3. **Acción registrada** en `ACTION_REGISTRY.keys()`.
4. **Acción dentro de `allowed_actions`**, si el manifest lo declara. Este control es valioso: impide publicar un flow cuya política se contradiga con sus pasos.
5. **`transitions.next` y `start_step` apuntan a pasos existentes**.

Devuelve JSON con `ok`, `flows_checked`, `registered_actions` y `errors[]`, y sale con código 1 si hay errores. Es el gate que corre la CI en las seis combinaciones de la matriz.

Lo que NO comprueba: que los `params` de cada paso coincidan con la firma de la acción. Un `screen.capture_screenshot` sin `output_path` pasa la validación y falla en ejecución con `TypeError`. Registrado en [12](#).

Documentos relacionados: [03](#) · [Arquitectura](#) · [04](#) · [Mapa del código](#) · [05](#) · [Referencia técnica](#) · [08](#) · [Flujo de datos](#) · [15](#) · [Riesgos](#)